

Static Code Analysis (SAST) with Semgrep

Course: Secure Software Development

Student Name: Jerome Arsany

GitHub Repository: <https://github.com/jeromearsany/Mutillidae-SAST-Analysis>

1. Introduction:

In this assignment, I used a tool called **Semgrep CLI**. This is a "Static Application Security Testing" (SAST) tool. Its job is to scan source code to find security mistakes without actually running the application.

I analyzed the **OWASP Mutillidae II** project, which is a website made with intentional security flaws. My goal was to stop looking for bugs manually and instead write **automated rules** to find them for me.

I focused on finding two specific types of dangerous vulnerabilities:

1. **Cross-Site Scripting (XSS)**
 2. **SQL Injection (SQLi)**
-

2. Reconnaissance (Finding the Vulnerabilities):

Before I could write the automation rules, I had to look at the code manually to understand how it was written. I searched the source code and found two major issues.

2.1 Cross-Site Scripting (XSS):

I found an issue in the file `"src/index.php"`.

The Problem: The code uses the PHP command `echo` to print user data directly to the screen. It does not use any cleaning functions (like `htmlspecialchars`).

Why it is dangerous: This allows an attacker to send a malicious script to the website, which will then run in the victim's browser.

Figure 1: The vulnerable XSS code found in index.php

```
echo $RemoteFileHandler->getNoCurlAdviceBasedOnOperatingSystem();
```

2.2 SQL Injection (SQLi):

I found a critical issue in the file `"src/classes/SQLQueryHandler.php"`.

The Problem: The code writes database queries by pasting variables directly into the command string (for example: `SELECT * ... '$username'`).

Why it is dangerous: An attacker can trick the database into revealing hidden password data or deleting files by changing the variable content.

Figure 2: The vulnerable SQL code found in SQLQueryHandler.php

```
$lQueryString = "
    SELECT  level_1_help_include_files.level_1_help_include_file_key,
            level_1_help_include_files.level_1_help_include_file_description
    FROM    page_help
    INNER JOIN level_1_help_include_files
    ON      page_help.help_text_key =
            level_1_help_include_files.level_1_help_include_file_key
    WHERE   page_help.page_name = ' . $pPageName . ' ' .
    "ORDER BY page_help.order_preference";
```

3. Creating Custom Rules:

I created a file named "**sast-custom-rules.yaml**". In this file, I wrote instructions for Semgrep to tell it exactly what "bad code" looks like.

Rule 1: SQL Injection Detection

- **ID:** sast-sqli-check
- **Logic:** I told Semgrep to look for any line of code that starts with "**SELECT**" and contains a PHP variable (\$VAR) inside the quotes.
- **Severity:** I marked this as **ERROR** because database attacks are very critical.

Rule 2: XSS Detection

- **ID:** sast-xss-check
- **Logic:** I told Semgrep to look for the command "**echo \$VAR**". However, I also added a "pattern-not" rule. This tells Semgrep: "If the code uses htmlspecialchars, ignore it. Only alert me if the data is raw."
- **Severity:** I marked this as **WARNING**.

Figure 3: My custom YAML rules file

```
rules:
- id: sast-sqli-check
  patterns:
  - pattern: |
    $query = "SELECT ... $VAR ... ";
  - pattern-not: |
    $query = "SELECT ... " . $mysqli->real_escape_string( ... ) . " ... ";
  message: "SAST ALERT: Possible SQL Injection. Variable used directly in SQL."
  languages: [php]
  severity: ERROR

- id: sast-xss-check
  patterns:
  - pattern: echo $VAR;
  - pattern-not: echo htmlspecialchars( ... );
  message: "SAST ALERT: Possible XSS. Variable echoed without HTML encoding."
  languages: [php]
  severity: WARNING
```

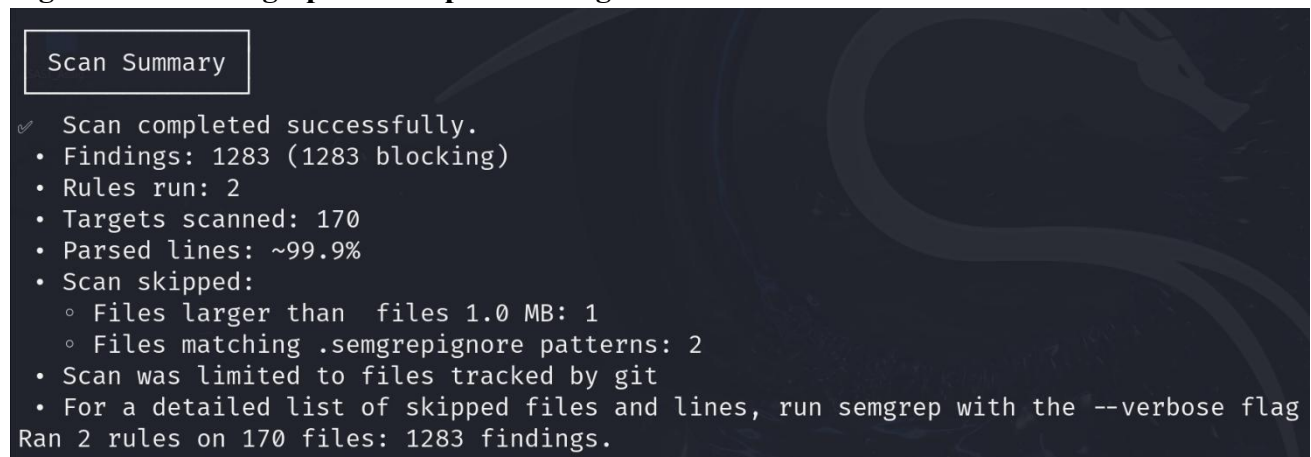
4. Scan Results:

After creating the rules, I ran the scanner using the following command in the terminal:

```
"semgrep --config sast-rules/sast-custom-rules.yaml src"
```

The scan was successful. Semgrep scanned the files and alerted me to both issues I found earlier. This proves that my rules work correctly.

Figure 4: The Semgrep scan output showing the detected errors



```
Scan Summary
✓ Scan completed successfully.
• Findings: 1283 (1283 blocking)
• Rules run: 2
• Targets scanned: 170
• Parsed lines: ~99.9%
• Scan skipped:
  ◦ Files larger than files 1.0 MB: 1
  ◦ Files matching .semgrepignore patterns: 2
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 2 rules on 170 files: 1283 findings.
```

5. Git Version Control:

To follow professional software development standards, I did not just work on my local computer. I tracked my changes using **Git**.

I created a specific branch named "**semgrep/jerome-arsany**". I committed my rule file and my documentation to this branch and pushed it to GitHub.

Figure 5: Git log proof showing my name and commit message



```
(kali@kali)~[~/Desktop/SAST_Assignment/mutillidae]
$ git log
commit 4966c8e992bd43f56a8a13f86a1b54128580d566 (HEAD -> semgrep/jerome-arsany, origin/semgrep/jerome-arsany)
Author: jeromearsany <jerome.arsany12@gmail.com>
Date: Mon Nov 24 16:58:23 2025 -0500

    Updated project with comprehensive README and final rules

commit d49a8adb37812ec0ee627b6399f55cb8947827fb
Author: jeromearsany <jerome.arsany12@gmail.com>
Date: Mon Nov 24 16:48:47 2025 -0500

    Added custom SAST rules and documentation
```

6. Conclusion:

This lab taught me the importance of **Automated Security Testing**.

Finding bugs by reading code manually takes a long time and is easy to mess up. By using **Semgrep**, I was able to write a rule once, and now I can scan the project anytime to catch these bugs instantly. This methodology helps developers build secure software much faster.